Multi-Pipeline ETL and Reporting Framework

for Web Server Log Analytics

Course: **DAS 839 – NoSQL Systems**

Phase: **Phase 1 — Architecture & Prototype**

Date: **28 April 2025**

Group: **Group 10**

Members: **Siddharth Anil (IMT2023503)**
Yash Gupta (IMT2023125)
Hitanshu Seth (IMT2023100)
Pranay Kelotra (IMT2023563)

Execution Backends: **MapReduce** **MongoDB** **Hive** **Pig**

GitHub Repository: `github.com/pkelotra/NO_SQL-Project`

NASA HTTP Web Server Logs — Kennedy Space Center (Jul–Aug 1995)

Contents

1 Overall Design of the Tool **2**
- 1.1 System Philosophy . . . 2
- 1.2 System Architecture Diagram . . . 2
- 1.3 Component Overview . . . 2

2 User Interface **3**
- 2.1 Control Panel UI . . . 3
- 2.2 Reporting (Yet to be Finalized) . . . 3

3 Batching **3**
- 3.1 Definition . . . 4
- 3.2 Batch Processing Diagram . . . 4

4 Parsing Strategy **4**
- 4.1 Log Format . . . 4
- 4.2 Extracted Fields . . . 5
- 4.3 Regex-Based Parsing . . . 5
- 4.4 Parsing Constraints . . . 5
- 4.5 Parsing Flow Diagram . . . 6

5 ETL Workflow **6**
- 5.1 Three Phases . . . 6
  - 5.1.1 Extract . . . 6
  - 5.1.2 Transform . . . 6
  - 5.1.3 Load . . . 6
- 5.2 ETL Flow Diagram . . . 7

6 Pipeline Overview **7**
- 6.1 MapReduce . . . 7
- 6.2 MongoDB . . . 7
- 6.3 Hive . . . 8
- 6.4 Pig . . . 8

7 Relational Database Schema **8**
- 7.1 Design Rationale . . . 8
- 7.2 Metadata Table: `run_metadata` . . . 8
- 7.3 Result Table 1: `daily_traffic` . . . 9
- 7.4 Result Table 2: `top_resources` . . . 9
- 7.5 Result Table 3: `hourly_errors` . . . 9

8 Ensuring Equivalence Across Pipelines **9**

9 Diagram Index **10**

10 Phase 1 Prototype Status **10**

1 Overall Design of the Tool

1.1 System Philosophy

The framework is designed as a **unified, end-to-end ETL and reporting tool** — not a collection of independent scripts. A single orchestration layer manages pipeline selection, dataset ingestion, transformation, result storage, and reporting. Swapping the execution backend (MapReduce / MongoDB / Hive / Pig) does not alter any other layer of the system; only the processing engine changes.

1.2 System Architecture Diagram

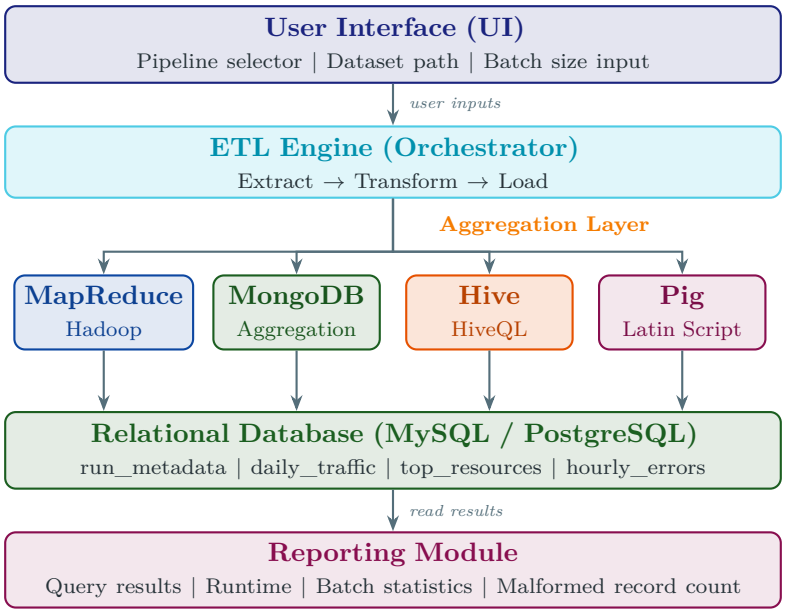


Figure 1 System architecture: the unified framework from UI through to the Reporting Module

1.3 Component Overview

The system comprises five tightly coupled components:

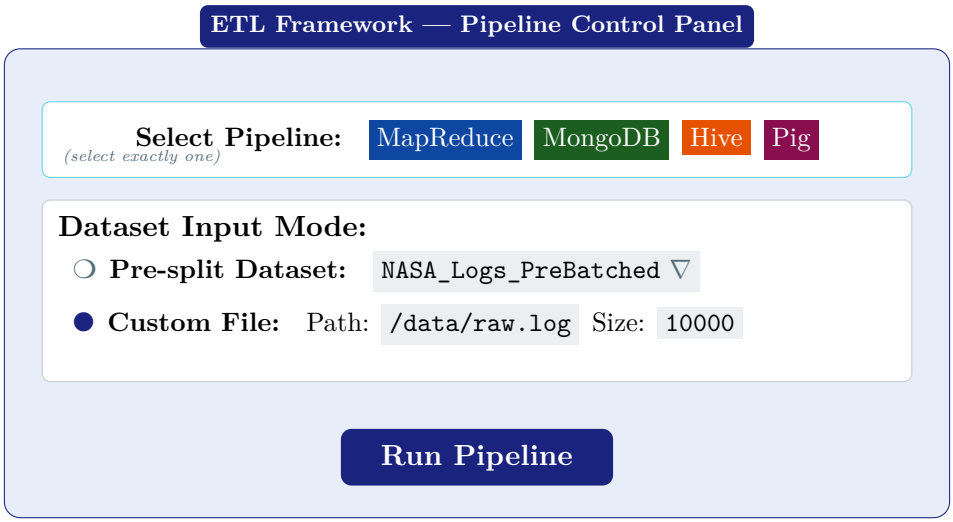
Component	Responsibility
User Interface (UI)	Accepts pipeline selection (one of four systems), dataset path, and batch size. Triggers the ETL engine and passes control to the Reporting Module on completion.
ETL Engine	Orchestrates Extract–Transform–Load. Reads raw logs, invokes the chosen service for parsing and aggregation, and pushes results to the relational database.
Aggregation Layer	The backend-specific execution layer (MapReduce / MongoDB / Hive / Pig) that computes the three mandatory queries over the structured records.
Relational Database	PostgreSQL store holding all query results. Acts as the single source of truth for cross-pipeline comparison.
Reporting Module	Reads the relational database and presents results in structured format.

2 User Interface

2.1 Control Panel UI

The UI is a command-line or lightweight web front-end that presents the operator with pipeline and dataset configurations before any computation begins. The user can select the target pipeline and choose between two data input modes:

- **Pre-split Dataset:** The user selects from a predefined list of dataset names that have already been partitioned into batches.
- **Custom Configuration:** The user manually specifies the path to a raw dataset file and the desired batch size. Upon execution, the system will dynamically split the file into batches before processing.



After execution, the UI passes control to the Reporting Module, which reads the relational database and renders structured results including query output tables, wall-clock runtime, batch statistics, and the malformed-record count.

2.2 Reporting (Yet to be Finalized)

The Reporting Module reads from the relational database and presents:

- **Run metadata:** pipeline name, run ID, batch ID, batch size, total runtime (seconds), malformed count
- **Query 1 table** — Daily Traffic Summary (grouped by date and status code)
- **Query 2 table** — Top 20 Requested Resources
- **Query 3 table** — Hourly Error Analysis (grouped by date and hour)

> ★ **Note**
>
> The framework is a single coherent tool. The execution backend is a pluggable module; all other components (UI, orchestration, database schema, reporting) remain identical across all four pipelines.

3 Batching

3.1 Definition

Batch Definition

Batch = a fixed number of input log records processed together in one execution unit.

- **Batch size** — configurable integer supplied via the UI (e.g. 10 000 records).
- **Batch ID** — sequential integer, incrementing by 1 from 1, for each subsequent batch.

The **average batch size** is reported as:

$$\text{batch size} = \frac{\text{total records processed}}{\text{number of non-empty batches}}$$

3.2 Batch Processing Diagram

Before records undergo parsing and aggregation, the framework orchestrates batching as dictated by the `PipelineRunner.java` driver. The system partitions the dataset based on the configured number of records per batch, and processes the queries sequentially.

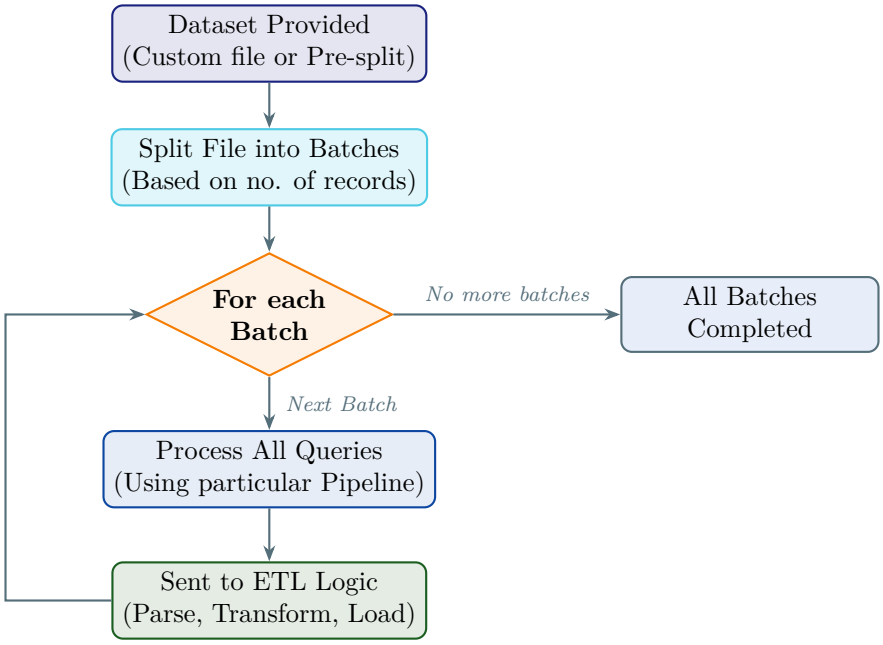


Figure 2 Flow chart showing dataset splitting and query execution per batch.

4 Parsing Strategy

4.1 Log Format

Each line in the NASA HTTP log follows the Combined Log Format. A representative record is:

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
```

4.2 Extracted Fields

Raw value	Extracted field	*Notes*
`199.72.81.55`	host	*IP address or hostname*
`01/Jul/1995:00:00:01 -0400`	timestamp	*Full timestamp string*
`01/Jul/1995`	log_date	*Date portion extracted from timestamp*
`00`	log_hour	*Hour portion (0–23)*
`GET`	http_method	*HTTP Method*
`/history/apollo/`	resource_path	*Target URL path requested by the client.*
`HTTP/1.0`	protocol_version	*HTTP version protocol string.*
`200`	status_code	*Integer HTTP reply code*
`6245`	bytes_transferred	*Integer; `-` is treated as 0*

4.3 Regex-Based Parsing

A robust master regular expression (`java.util.regex.Pattern`) captures all eight required fields in one pass. Instead of capturing the entire timestamp string, targeted capture groups selectively extract the `logDate` and `logHour` directly. The `bytes` capture group enforces strict pattern validation by only matching digits or a hyphen.

Listing 1 Master regular expression for log-line parsing

```
^(\S+)\s+\S+\s+\S+\s+\[(\d{2}/\w{3}/\d{4}):(\d{2}):\d{2}:\d{2}\s
    +[+-]\d{4}\]\s+"([A-Z]+)\s+(\S+)\s+(HTTP/[\d.]+)"\s+(\d{3})\s+(\d
    +|-)$
```

Capture-group mapping: `(1) host`, `(2) logDate`, `(3) logHour`, `(4) method`, `(5) resource`, `(6) protocol`, `(7) statusCode`, `(8) bytes_raw`.

The `bytes` field is derived as:

$$\text{bytes} = \begin{cases} 0 & \text{if } \texttt{bytes_raw} = \texttt{"-"} \\ \text{int}(\texttt{bytes_raw}) & \text{otherwise} \end{cases}$$

4.4 Parsing Constraints

✘ Implementation Constraint

- If the bytes field contains `-`, it **must** be substituted with **0** (not discarded).
- Malformed records (lines that do not match the master regex) must **not** be silently dropped. A `malformed_record_counter` is incremented and the count is persisted to the metadata table at the end of each run.

4.5 Parsing Flow Diagram

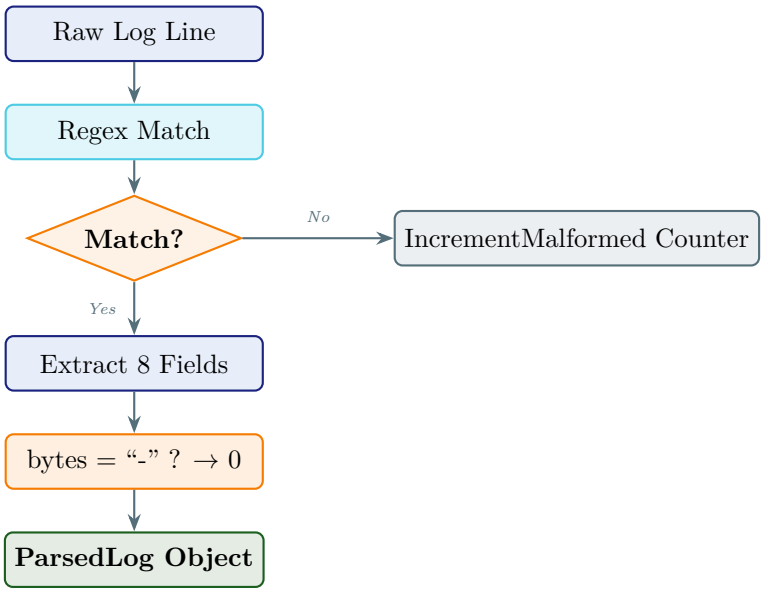


Figure 3 Parsing flow: from raw log line to validated ParsedLog object

5 ETL Workflow

The ETL workflow remains logically identical across the implemented execution pipelines (MapReduce and MongoDB), ensuring consistent methodology regardless of the underlying engine.

5.1 Three Phases

5.1.1 Extract

- Raw log files are read from the path supplied by the user through the UI. No preprocessing outside the chosen execution pipeline is permitted (no manual CSV conversion, JSON conversion, filtering, or editing). Files are read line by line and dispatched to the transform stage in configurable batches.
- The `PipelineRunner.java` supplies the dataset path and generates a unique SQL `runId` via `MetadataDAO.java`.

5.1.2 Transform

- **Parsing:** The master regex ,i.e, `LogParser.java` regex validates and extracts the 8 core fields line-by-line.
- **Cleaning & Tracking:** Hyphens (`-`) in bytes are safely converted to `0`. Malformed records dynamically increment a tracking counter and are excluded from calculations.
- **Query Aggregation:** The engine executes grouping and arithmetic logic (e.g., counting requests, summing bytes, distinct hosts).

5.1.3 Load

- **Result Retrieval:** The three mandatory queries are executed on the structured data within the chosen backend.
- **Database Insertion:** Structured results are persisted directly to PostgreSQL via Data Access Objects .
- **Finalization:** Run metadata (runtime, malformed count, batch statistics) is inserted into the `run_metadata` table.

5.2 ETL Flow Diagram

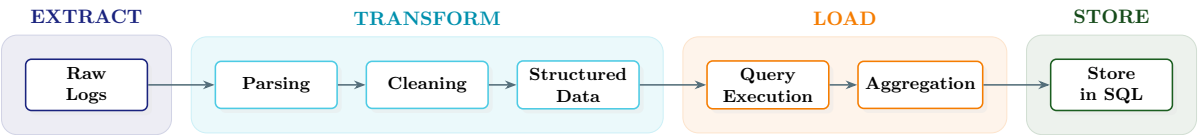


Figure 4 The unified ETL Workflow: consistent processing across all four pipelines.

★ Note

The parsing rules, cleaning rules, field definitions, query definitions, and output schema are strictly identical across MapReduce, MongoDB, Hive, and Pig. This ensures cross-pipeline comparisons are fair and meaningful.

6 Pipeline Overview

Each pipeline executes the same three analytical queries using a different processing paradigm. All four implement equivalent semantics: the same parsing rules, cleaning rules, filtering conditions, grouping logic, and aggregation definitions.

6.1 MapReduce

Paradigm: Low-level, programmer-controlled parallel computation on HDFS.
Execution Model:

- **Mapper:** Reads raw HDFS splits and applies `LogParser.java`. Valid records emit composite `Text` keys (e.g., date & hour) and values (e.g., status & host). It tracks data quality dynamically using Hadoop Counters (`MALFORMED` vs `VALID_RECORDS`).
- **Reducer:** Aggregates grouped keys iteratively. For example, `HourlyErrorMR` loops through values to count total requests, isolate errors (status 400–599), and uses a Java `HashSet` for distinct host tracking before calculating the final error rate.
- **Driver:** Configures the job, triggers execution, reads the final HDFS `part-*` output files sequentially using a `BufferedReader`, and commits the aggregated results to PostgreSQL via DAOs (e.g., `Q3DAO.saveResult()`) before cleaning up the temporary HDFS directory.

6.2 MongoDB

Paradigm: Document-oriented NoSQL with a declarative aggregation framework.
Execution Model:

- **Ingestion:** The driver uses a `BufferedReader` to process the local file line-by-line via `LogParser.java`. Valid lines are converted into BSON `Document` objects and loaded into the database using an efficient `collection.insertMany()` bulk operation.
- **Aggregation:** Queries are executed via the MongoDB Java Driver's declarative aggregation pipeline (`collection.aggregate()`). For example, `Q1Mongo` uses `Aggregates.group()` combined with `Accumulators.sum()` to compute request counts and total bytes directly within the database engine.
- **Result Extraction:** An `AggregateIterable<Document>` cursor iterates over the com-

puted BSON outputs, extracts the metrics, and delegates permanent storage to PostgreSQL via DAOs (e.g., `Q1DAO.saveResult()`).

6.3 Hive

Paradigm: SQL-like query language (HiveQL) over Hadoop.
Execution Model: *[To be updated based on Hive implementation]*

- **Ingestion:** ...
- **Aggregation:** ...
- **Result Extraction:** ...

6.4 Pig

Paradigm: Dataflow scripting language (Pig Latin) that compiles to MapReduce.
Execution Model: *[To be updated based on Pig implementation]*

- **Ingestion:** ...
- **Aggregation:** ...
- **Result Extraction:** ...

7 Relational Database Schema

7.1 Design Rationale

The schema separates run-level metadata from query results. One metadata table stores information common to all queries (pipeline name, runtime, batch info). Three result tables—one per mandatory query—hold the aggregated outputs and reference the metadata table via `run_id` (foreign key). This design allows any run's results to be retrieved, filtered by pipeline or batch, and compared against other runs.

7.2 Metadata Table: `run_metadata`

Column	Type	Description
`run_id`	SERIAL PK	Auto-incremented unique identifier for each run
`pipeline_name`	VARCHAR(20)	Execution engine (e.g., `mapreduce`, `mongodb`)
`batch_id`	INTEGER	Sequential batch identifier (starts at 1)
`batch_size`	INTEGER	Configured number of records per batch
`avg_batch_size`	FLOAT	Calculated average batch size across the dataset
`runtime`	FLOAT	Wall-clock time in seconds (ETL only)
`malformed_record_count`	INTEGER	Number of log lines that failed regex parsing
`dataset_name`	VARCHAR	The name or path of the selected dataset
`timestamp`	TIMESTAMP	UTC datetime when the run was recorded

7.3 Result Table 1: `daily_traffic`

Stores the output of Query 1 (Daily Traffic Summary).

Column	Type	Description
`id`	SERIAL PK	Row identifier
`run_id`	INTEGER FK	References `run_metadata.run_id`
`log_date`	DATE	Date of the log entries (e.g. 01/Jul/1995)
`status_code`	INTEGER	HTTP reply code
`request_count`	BIGINT	Total number of requests for this date–code pair
`total_bytes`	BIGINT	Sum of bytes transferred for this date–code pair

7.4 Result Table 2: `top_resources`

Stores the output of Query 2 (Top Requested Resources). Only the top 20 rows by request count are stored per run.

Column	Type	Description
`id`	SERIAL PK	Row identifier
`run_id`	INTEGER FK	References `run_metadata.run_id`
`resource_path`	TEXT	Requested URL path
`request_count`	BIGINT	Number of times this resource was requested
`total_bytes`	BIGINT	Total bytes served for this resource
`distinct_host_count`	BIGINT	Number of unique hosts requesting this resource

7.5 Result Table 3: `hourly_errors`

Stores the output of Query 3 (Hourly Error Analysis).

Column	Type	Description
`id`	SERIAL PK	Row identifier
`run_id`	INTEGER FK	References `run_metadata.run_id`
`log_date`	DATE	Date of the log entries
`log_hour`	SMALLINT	Hour of day (0–23)
`error_request_count`	BIGINT	Requests with status code 400–599
`total_request_count`	BIGINT	All requests in this date–hour slot
`error_rate`	FLOAT	`error_request_count / total_request_count`
`distinct_error_hosts`	BIGINT	Unique hosts generating error responses

8 Ensuring Equivalence Across Pipelines

To make the cross-pipeline comparison fair and meaningful, the following invariants are enforced throughout the implementation:

1. **Same dataset.** All pipelines read the identical NASA HTTP log files downloaded from the official Internet Traffic Archive. No manual preprocessing, CSV conversion, JSON conversion, filtering, or editing is applied outside the selected pipeline.

2. **Same parsing rules.** The master regex and field-extraction logic are defined once at the architecture level and translated faithfully into each backend (Mapper code, MongoDB UDF, Hive SerDe or regex UDF, Pig LOAD function with UDF).

3. **Same query definitions.** The three mandatory queries specify exact output columns, filtering conditions (e.g. status 400–599 for errors), and aggregation semantics. These are not relaxed or simplified for any particular pipeline.
4. **Same batch size.** In any comparative experiment, the same batch-size value is supplied to all four pipelines via the shared UI.
5. **Same output schema.** All results are written to the shared relational database in an identical schema regardless of which backend produced them. This allows direct SQL comparison across runs.

9 Diagram Index

Figure	Section	Description
1	**1**	System Architecture — UI through to Reporting Module
2	**5**	Batch Processing — Input split into k equal batches
3	**2**	Parsing Flow — Raw line to validated structured record
4	**3**	ETL Workflow — Extract, Transform, Load

10 Phase 1 Prototype Status

At the time of this Phase 1 review, the following components are complete or in active development:

Component	Status	Applies to
UI (pipeline selector, dataset path, batch size)	✔ In Progress	All
Master regex parser + malformed counter	✔ Done	All
MapReduce ETL (Q1, Q2, Q3)	✔ Done	MapReduce
MongoDB ETL (Q1, Q2, Q3)	✔ Done	MongoDB
Hive ETL	▲ In Progress	Hive
Pig ETL	▲ In Progress	Pig
Relational database schema and load scripts	✔ Done	All
Reporting module	✔ In Progress	All

Phase 2 will focus on finalizing the **UI and Reporting modules** alongside the completion of the **Hive and Pig pipelines**. Once the framework is fully integrated, the focus will shift to executing comparative experiments across all four backends using standardized batch sizes. This phase will conclude with a comprehensive documentation of runtime observations, implementation trade-offs, and cross-pipeline correctness verification.